AIML LAB ASSIGNMENT

Name: Prachi Lohkare
Div: A Batch: A1 Roll no: A1-04
PRN: 202401040025
Colab Link: _____

Performance Evaluation of Regression Model

Problem Statement

Develop a regression model for a real-world application and evaluate its performance using suitable metrics. The Electric Vehicle (EV) dataset is used to model real-world driving range from battery capacity. A linear regression model is trained, evaluated, and compared with a manually implemented Gradient Descent optimization method.

Setup

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Dataset

```
uploaded = files.upload()

file_path = next(iter(uploaded))
df = pd.read_csv(file_path)

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
display(df.head())
print("\nColumn Names:")
print(list(df.columns))
```

```
Rows: 7291
Columns: 25

Column Names:
['make', 'model', 'variant', 'battery_capacity_kWh', 'battery_capacity_kWh_new', 'motor_power_kW', 'vehicle_weight_kg',
'drag_coefficient_Cd', 'frontal_area_m2', 'rolling_resistance_Cr', 'motor_efficiency_pct', 'ARAI_certified_range_km',
'avg_driving_speed_kmh', 'ambient_temperature_C', 'ac_usage', 'terrain_type', 'traffic_condition', 'driving_style', 'passengers',
'battery_age_years', 'tyre_pressure_deviation_psi', 'city_road_type', 'energy_consumption_kWh_per_km', 'real_world_range_km',
'price_lakh_INR']
```

	make	model	variant	battery_capacity_kWh	battery_capacity_kWh_new	motor_power_kW	vehicle_weight_kg	drag_coefficient_Cd	frontal_area_m2	rolling_resistance
0	Tata	Nexon EV	Long Range	35.3	40.5	105	1610	0.34	2.4	0.0122
1	Tata	Nexon EV	Long Range	40.5	40.5	105	1610	0.34	2.4	0.0120
2	Tata	Nexon EV	Long Range	39.3	40.5	105	1610	0.34	2.4	0.0122
3	Tata	Nexon EV	Long Range	39.0	40.5	105	1610	0.34	2.4	0.0120
4	Tata	Nexon EV	Long Range	37.9	40.5	105	1610	0.34	2.4	0.0120

Select Variables

Set the two names below according to the columns shown above.

```
# Change only these two names if your CSV uses different column names.
X_column = 'battery_capacity_kWh'
y_column = 'real_world_range_km'

data = df[[X_column, y_column]].copy()

print("Selected input:", X_column)
print("Selected target:", y_column)
```

```
Selected input: battery_capacity_kWh
Selected target: real_world_range_km
```

Cleaning

```
data[X_column] = pd.to_numeric(data[X_column], errors='coerce')
data[y_column] = pd.to_numeric(data[y_column], errors='coerce')

invalid_battery = int((data[X_column] <= 0).sum())
invalid_range = int((data[y_column] <= 0).sum())

data.loc[data[X_column] <= 0, X_column] = np.nan
data.loc[data[y_column] <= 0, y_column] = np.nan

before = len(data)
data = data.dropna().reset_index(drop=True)

print("Rows before cleaning:", before)
print("Invalid battery capacity values:", invalid_battery)
print("Invalid range values:", invalid_range)
print("Rows removed:", before - len(data))
print("Rows after cleaning:", len(data))
display(data.head(10))
```

```
Rows before cleaning: 7291
Invalid battery capacity values: 0
Invalid range values: 0
Rows removed: 0
Rows after cleaning: 7291
```

	battery_capacity_kWh	real_world_range_km
0	35.3	141.00
1	40.5	338.00
2	39.3	313.40
3	39.0	441.75
4	37.9	193.40
5	38.8	280.40
6	37.9	142.60
7	38.4	329.80
8	34.5	194.50
9	36.6	256.70

Split

```
X = data[[X_column]]
y = data[y_column]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 5832
Testing samples: 1459
```

Linear Regression

```
model = LinearRegression()
model.fit(X_train, y_train)

sk_b = model.intercept_
sk_w = model.coef_[0]
y_pred = model.predict(X_test)

print(f"Intercept: {sk_b:.6f}")
print(f"Slope: {sk_w:.6f}")
print(f"Regression Equation: EV Range = {sk_b:.2f} + ({sk_w:.2f} × Battery Capacity)")
```

```
Intercept: 110.858939
Slope: 2.228810
Regression Equation: EV Range = 110.86 + (2.23 × Battery Capacity)
```

Gradient Descent

```
def gradient_descent(X, y, learning_rate=0.01, epochs=5000):
    x = np.asarray(X, dtype=float).reshape(-1)
    target = np.asarray(y, dtype=float).reshape(-1)

    valid = np.isfinite(x) & np.isfinite(target)
    x = x[valid]
    target = target[valid]

    # Standardize the input so Gradient Descent converges quickly
    x_mean, x_std = x.mean(), x.std()
    x = (x - x_mean) / x_std

    n = len(x)
    w = 0.0
    b = 0.0
    history = []

    for _ in range(epochs):
        pred = w * x + b
        error = pred - target

        history.append(np.mean(error ** 2))

        dw = (2 / n) * np.sum(x * error)
        db = (2 / n) * np.sum(error)

        w -= learning_rate * dw
        b -= learning_rate * db

    # Convert parameters back to the original battery-capacity scale
    w_orig = w / x_std
    b_orig = b - w * x_mean / x_std
    return w_orig, b_orig, np.array(history)
```

```
gd_w, gd_b, loss_history = gradient_descent(
    X_train[X_column], y_train, 0.01, 5000
)

gd_pred = gd_w * X_test[X_column].to_numpy() + gd_b

print("Learning rate: 0.01")
print("Epochs: 5000")
print(f"Intercept: {gd_b:.6f}")
print(f"Slope: {gd_w:.6f}")
print(f"Final training MSE: {loss_history[-1]:.6f}")
```

```
Learning rate: 0.01
Epochs: 5000
Intercept: 110.858939
Slope: 2.228810
Final training MSE: 19078.576839
```

Metrics

```
metrics = pd.DataFrame({
    'MAE': [
        mean_absolute_error(y_test, y_pred),
        mean_absolute_error(y_test, gd_pred)
    ],
    'MSE': [
        mean_squared_error(y_test, y_pred),
        mean_squared_error(y_test, gd_pred)
    ],
    'RMSE': [
        np.sqrt(mean_squared_error(y_test, y_pred)),
        np.sqrt(mean_squared_error(y_test, gd_pred))
    ],
    'R2': [
        r2_score(y_test, y_pred),
        r2_score(y_test, gd_pred)
    ]
}, index=['Linear Regression', 'Gradient Descent'])

display(metrics.round(4))
```

	MAE	MSE	RMSE	R2
Linear Regression	113.8566	18636.0139	136.5138	0.0913
Gradient Descent	113.8566	18636.0139	136.5138	0.0913

Predictions

```
predictions = pd.DataFrame({
    'Battery Capacity (kWh)': X_test[X_column].to_numpy(),
    'Actual EV Range (km)': y_test.to_numpy(),
    'Linear Regression': y_pred.round(2),
    'Gradient Descent': gd_pred.round(2)
}).sort_values('Battery Capacity (kWh)')

print('Total test predictions:', len(predictions))
display(predictions.head(15))
```

```
Total test predictions: 1459
```

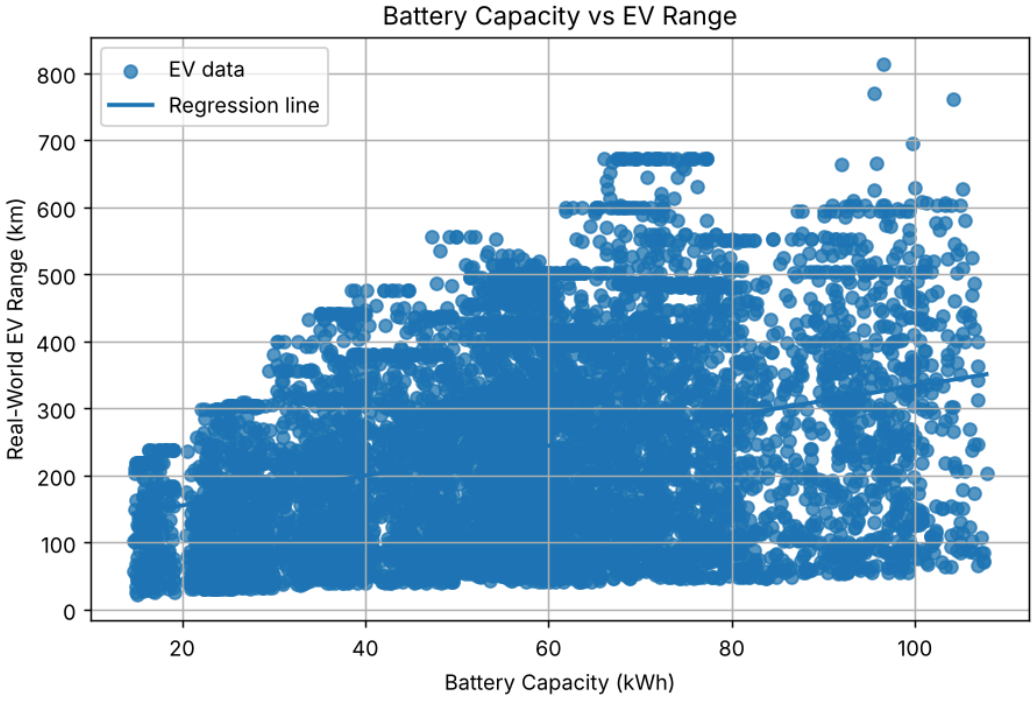
	Battery Capacity (kWh)	Actual EV Range (km)	Linear Regression	Gradient Descent
348	15.0	52.3	144.29	144.29
928	15.0	162.5	144.29	144.29
950	15.1	78.3	144.51	144.51
1115	15.2	218.5	144.74	144.74
172	15.2	124.1	144.74	144.74
248	15.2	218.5	144.74	144.74
213	15.5	50.8	145.41	145.41
861	15.5	209.1	145.41	145.41
204	15.6	150.9	145.63	145.63
655	15.7	28.5	145.85	145.85
13	16.0	193.3	146.52	146.52
278	16.1	154.9	146.74	146.74
86	16.1	80.7	146.74	146.74
937	16.4	237.5	147.41	147.41
912	16.4	147.1	147.41	147.41

Regression Plot

```
plt.figure(figsize=(8, 5))
plt.scatter(data[X_column], data[y_column], alpha=0.75, label='EV data')

x_line = np.linspace(data[X_column].min(), data[X_column].max(), 200)
plt.plot(x_line, sk_b + sk_w * x_line, linewidth=2, label='Regression line')

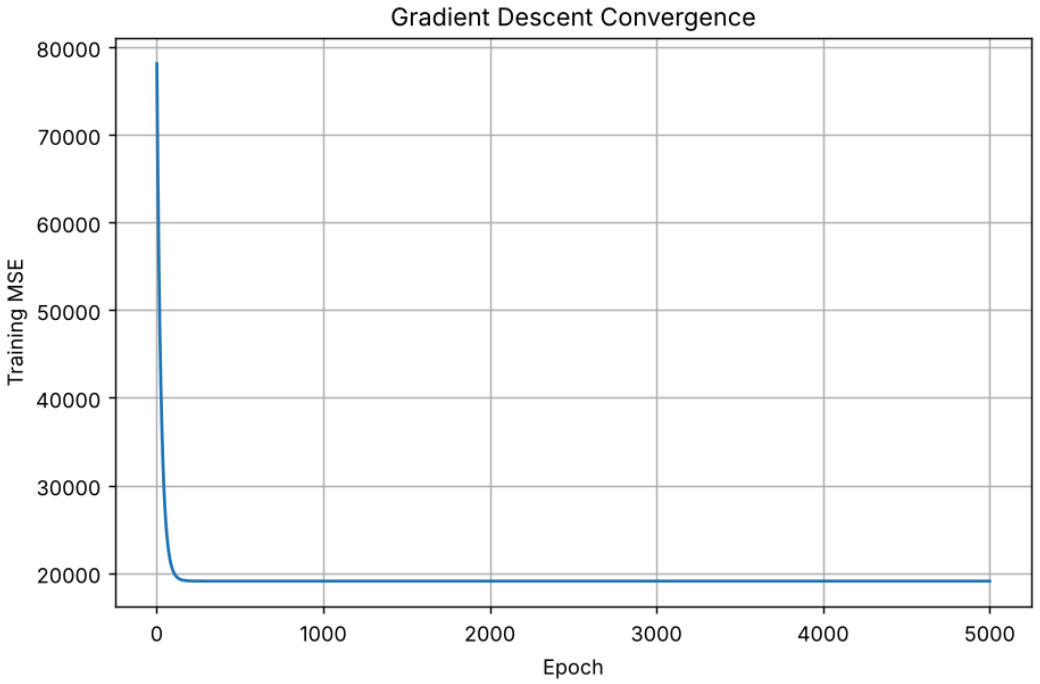
plt.xlabel('Battery Capacity (kWh)')
plt.ylabel('Real-World EV Range (km)')
plt.title('Battery Capacity vs EV Range')
plt.legend()
plt.grid(True)
plt.show()
```



Convergence

```
plt.figure(figsize=(8, 5))
plt.plot(np.arange(1, len(loss_history) + 1), loss_history)

plt.xlabel('Epoch')
plt.ylabel('Training MSE')
plt.title('Gradient Descent Convergence')
plt.grid(True)
plt.show()
```



Learning Rate

```
learning_rates = [0.001, 0.005, 0.01]
lr_results = []

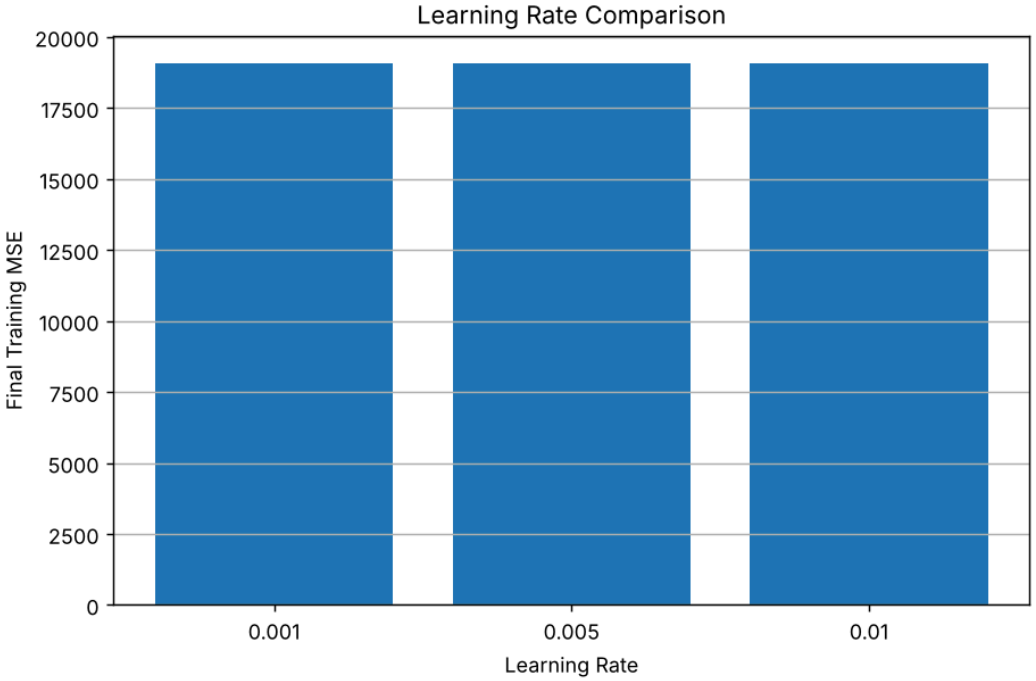
for rate in learning_rates:
    w, b, history = gradient_descent(
        X_train[X_column], y_train, rate, 5000
    )
    lr_results.append([rate, 5000, w, b, history[-1]])

lr_results = pd.DataFrame(
    lr_results,
    columns=['Learning Rate', 'Epochs', 'Slope', 'Intercept', 'Final Train MSE']
)

display(lr_results.round(6))

plt.figure(figsize=(8, 5))
plt.bar([str(r) for r in lr_results['Learning Rate']], lr_results['Final Train MSE'])
plt.xlabel('Learning Rate')
plt.ylabel('Final Training MSE')
plt.title('Learning Rate Comparison')
plt.grid(True, axis='y')
plt.show()
```

	Learning Rate	Epochs	Slope	Intercept	Final Train MSE
0	0.001	5000	2.22871	110.853956	19078.576959
1	0.005	5000	2.22881	110.858939	19078.576839
2	0.010	5000	2.22881	110.858939	19078.576839

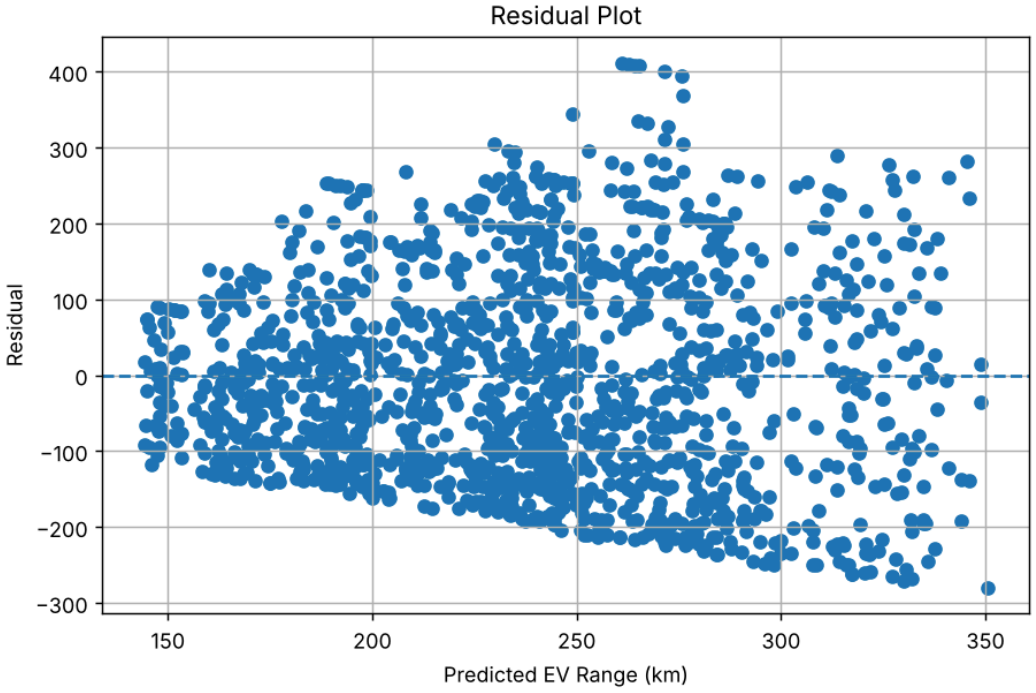


Residuals

```
residuals = y_test.to_numpy() - y_pred

plt.figure(figsize=(8, 5))
plt.scatter(y_pred, residuals)
plt.axhline(0, linestyle='--')

plt.xlabel('Predicted EV Range (km)')
plt.ylabel('Residual')
plt.title('Residual Plot')
plt.grid(True)
plt.show()
```



New Prediction

```
new_data = pd.DataFrame({X_column: [50]})
prediction = model.predict(new_data)[0]

print(f"Predicted EV range for 50 kWh battery: {prediction:.2f} km")
print(
    "Parameters close:",
    np.isclose(sk_w, gd_w, atol=0.001)
    and np.isclose(sk_b, gd_b, atol=0.001)
)
```

```
Predicted EV range for 50 kWh battery: 222.30 km
Parameters close: True
```

Comparison

```
print("Comparison")
display(metrics.round(4))
```

Linear Regression and the manually implemented Gradient Descent converge to identical parameters and produce matching test metrics, confirming the Gradient Descent implementation is correct.

```
Comparison
```

	MAE	MSE	RMSE	R2
Linear Regression	113.8566	18636.0139	136.5138	0.0913
Gradient Descent	113.8566	18636.0139	136.5138	0.0913

Result

```
print(f"Regression equation: EV Range = {sk_b:.2f} + ({sk_w:.2f} × Battery Capacity)")
print(f"Test R2 = {r2_score(y_test, y_pred):.4f}")
print(f"Prediction for 50 kWh battery = {prediction:.2f} km")
```

```
Regression equation: EV Range = 110.86 + (2.23 × Battery Capacity)
Test R2 = 0.0913
Prediction for 50 kWh battery = 222.30 km
```

Note: Test R^2 is low (≈0.09) because battery capacity alone explains little of the variation in real-world range. The dataset also depends on speed, temperature, AC usage, terrain, traffic and driving style, so adding those features would improve the model.